PROJECT WARREN
Structure guide

sep.2026

Collaborators:
Andrés Rodarte — sonderbot-ar

Project Scope and Vision 2
Core Architecture | [sep.23.2026] 2
1. Model Allocation: Local API Key Initiation 2
2. State Management: Central Database 2
3. Telemetry Integration: Local Log File 2
Project Structure | [sep.23.2026] 3

PROJECT WARREN

Project Scope and Vision

Project "Warren" is a custom-built, modular financial simulation engine powered by a multi-agent Large Language Model (LLM) architecture. The vision is to build a probabilistic stress-testing tool that merges deterministic financial calculations (e.g., Discounted Cash Flow models) with behavioral and qualitative variables (e.g., shifting geopolitical or academic sentiment).

The system allows users to alter tunable parameters—representing different real-world scenarios or sentiment shocks—and evaluate a company's or industry's financial performance under those specific conditions. To support continuous development, scalability, and deep observability, the codebase mandates an atomic design paradigm: every script and function must perform a single, indivisible step. The architecture is built from scratch utilizing LangGraph for orchestration and custom Python for quantitative execution, avoiding reliance on "black-box" third-party financial AI frameworks.

Core Architecture | [sep.24.2026]

1. Model Allocation: Local API Key Initiation

- **Decision:** API keys and model initializations will be declared locally within each specific domain's tools.py file, rather than globally.
- **Reasoning:** This allows the project to assign different LLM models to different tasks (e.g., a fast, cheap model for data scraping and a heavy reasoning model for financial orchestration). It guarantees strict node traceability, ensuring token billing and logic failures are isolated by their specific domain.

2. State Management: Central Database & EAV Model

- **Decision:** Instead of passing large data payloads directly between agents in active memory, nodes will read and write to a central database and pass lightweight reference IDs. The database utilizes an Entity-Attribute-Value (EAV) structure for financial metrics.
- **Reasoning:** A primary goal of Warren is to evaluate agent behavior and trace how agents interact with data. A central database provides a permanent record, allowing the telemetry system to identify exactly when, where, and how an agent

creates a data dependency. The EAV model allows infinite horizontal scaling of financial metrics without requiring hardcoded SQL migrations as agents extract new data points.

3. Multi-Tiered Data Processing

- **Decision:** Warren strictly separates deterministic quantitative data processing from unstructured qualitative/narrative processing.
- **Reasoning:** Pandas is highly efficient for time-series alignment and vectorized math but inefficient for unstructured text. To prevent memory bloat and data pipeline failures, quantitative EAV data feeds into Pandas DataFrames, while qualitative unstructured data (e.g., shareholder decks) is converted to semantic embeddings and housed in a ChromaDB vector database, triggered only on explicit user requests (deep-dives).

4. Mathematical Domain Isolation (Single Responsibility)

- **Decision:** Financial mathematical formulas are entirely decoupled from the data wrangling pipeline.
- **Reasoning:** To avoid code bloat and unmaintainable monoliths, the system uses a dedicated calculation library. math_engine.py operates exclusively as the data pipeline (fetching and pivoting DataFrames), while financial_math.py operates as a pure, deterministic calculation library. This enables isolated unit testing of financial formulas without requiring a mocked SQLite database or LangGraph agent state.

5. Telemetry Integration: Local Log File

- **Decision:** The decoupled observer system will initially record system health and token usage by appending JSON payloads to a local temporary log file, rather than broadcasting asynchronous API calls.
- **Reasoning:** Writing to a local file eliminates network-related bugs and reduces complexity during the initial build phase. Once the core LangGraph orchestrator and database logic are stable, this decoupled emitter can be easily swapped to an API-based system with minimal friction.

➢ ring the initial build phase. Once the core LangGraph orchestrator and database logic are stable, this decoupled emitter can be easily swapped to an API-based system with minimal friction.